

JEONME

Dokumentasi CI/CD & Deployment

*GitHub Actions + Docker + VPS — Pipeline, Audit, dan
Panduan Setup*

Dokumen	CICD-DOC v1.0 (konsolidasi CICD-GUIDE.md & SETUP-GUIDE.md)
Status	13 dari 14 temuan audit sudah diperbaiki
Tanggal	3 Juli 2026
Sumber	<code>.github/workflows/*.yaml</code> , <code>docker-compose*.yaml</code>

1. Filosofi Pipeline

- Setiap push ke `main` → build, test, lint otomatis. Gagal test = tidak lanjut deploy.
- Merge ke `main` (setelah PR di-approve) → auto-deploy ke **staging**.
- Tag rilis (`v1.0.0` , dst.) → deploy manual/terkontrol ke **production** (perlu approval).
- **Tidak pernah** deploy langsung dari laptop developer ke production — semua lewat pipeline agar konsisten & bisa diaudit.
- Docker image dibangun sekali di CI, image yang sama dipakai di staging maupun production.

BRANCH	DEPLOY OTOMATIS KE	CATATAN
<code>feature/*</code>	Tidak ada (hanya CI test)	Branch kerja harian developer
<code>develop</code>	(opsional) preview ringan	Integrasi fitur sebelum ke <code>main</code>
<code>main</code>	staging (otomatis)	Selalu mencerminkan kondisi siap-QA
Tag <code>v*.*.*</code>	production (manual approval)	Rilis resmi

2. Workflow CI — Test & Build (`ci.yml`)

Jalan di setiap push & pull request ke `main` / `develop` . Tidak melakukan deploy apa pun.

JOB	LANGKAH KUNCI
test-backend	Setup Go 1.25 → <code>go vet</code> → <code>go build</code> → terapkan migrasi ke DB test (<code>./bin/api migrate up</code>) → <code>go test ./... -v</code> → <code>govulncheck</code> (non-blocking)
test-frontend	Setup Node 24 → <code>npm ci</code> → lint → typecheck → build → <code>npm audit</code> (non-blocking)
build-images	Hanya jalan saat <code>push</code> (bukan PR) dan setelah dua job di atas sukses. Push image ke GHCR dengan tag <code>\${'{' }sha{' }'}</code> , plus tag <code>:latest</code> khusus dari branch <code>main</code> (fallback registry untuk deploy pertama kali)

3. Workflow Deploy — Staging (`deploy-staging.yml`)

Trigger: `workflow_run` setelah CI sukses di `main` . Urutan langkah (via SSH ke VPS staging):

1. **Login GHCR di VPS** — `docker login ghcr.io` pakai secret `GHCR_USERNAME` / `GHCR_PAT`
2. **Jalankan migrasi database** — pull image API baru, `docker compose run --rm api ./api migrate up`
3. **Deploy** — set `IMAGE_TAG` di `.env` , `docker compose pull && up -d --remove-orphans` , `docker image prune`
4. **Health check** — `curl` dengan retry berjeda ke `staging.jeonme.com/api/health`

4. Workflow Deploy — Production (`deploy-production.yml`)

Trigger: push tag `v*.*.*` atau manual (`workflow_dispatch`). Memakai GitHub **Environment** `production` dengan *required reviewers* — deploy tertahan menunggu approval manual. Urutan langkah sama seperti staging (login GHCR → migrasi → deploy → health check), dengan domain `jeonme.com` .

Wajib: endpoint `/api/health` mengecek koneksi DB + Redis (bukan cuma "container jalan") — lihat Dokumentasi Backend Bagian 7.1.

5. Ringkasan Audit CI/CD (3 Juli 2026)

14 temuan diverifikasi langsung terhadap kode (bukan asumsi dari dokumen); 13 sudah diperbaiki.

#	SEVERITY	TEMUAN	STATUS
C1	KRITIS	Migrasi production hanya placeholder <code>echo</code> , tidak pernah jalan sungguhan	Selesai
C2	KRITIS	Tidak ada <code>docker login ghcr.io</code> di VPS — pull GHCR privat akan gagal 401	Selesai
C3	KRITIS	CI tidak menerapkan migrasi sebelum test, belum ada test sama sekali	Selesai
H1	TINGGI	Tidak ada tag <code>:latest</code> fallback yang benar-benar ada di registry	Selesai
H2	TINGGI	Placeholder <code>OWNER</code> di compose files tidak pernah diganti	Selesai — diganti <code>\${'{' }GHCR_REPO{' }' }</code>
H3	TINGGI	Server block 443 tidak ada meski volume Certbot sudah dipasang	Selesai
M1	SEDANG	<code>npm install</code> dipakai, bukan <code>npm ci</code> (non-reproducible)	Selesai
M2	SEDANG	Health check deploy cuma <code>sleep 10</code> + satu kali <code>curl</code>	Selesai — retry berjeda
M3	SEDANG	Tidak ada pemindaian kerentanan di pipeline	Selesai — <code>govulncheck / npm audit non-blocking</code>
M4	SEDANG	<code>go.sum</code> belum ada di repo	Selesai
L1	RENDAH	CICD-GUIDE.md tidak sinkron dengan implementasi Go/Gin aktual	Selesai
L2	RENDAH	Tidak ada mekanisme rollback terprogram di GitHub Actions	Belum — mitigasi manual: <code>scripts/rollback.sh</code>
L3	RENDAH	Linting Go minim (hanya <code>go vet</code>)	Belum
L4	RENDAH	Worker/queue belum ada meski direncanakan async	Belum — dijadwalkan Sprint 2

6. Secrets & GitHub Environments

SECRET	ENVIRONMENT	ISI
STAGING_HOST / PROD_HOST	staging / production	IP atau hostname VPS
STAGING_SSH_USER / PROD_SSH_USER	staging / production	User deploy khusus (bukan root)
STAGING_SSH_KEY / PROD_SSH_KEY	staging / production	Private key SSH khusus deploy
GHCR_USERNAME	staging & production	Username GitHub pemilik token
GHCR_PAT	staging & production	Personal Access Token, scope <code>read:packages</code> saja
GITHUB_TOKEN	otomatis	Dipakai push image ke GHCR dari runner Actions

Di environment `production`, aktifkan **Required reviewers** (Settings → Environments) — minimal 1 approval sebelum deploy jalan.

7. Panduan Setup — 10 Fase

Ringkasan; instruksi lengkap dengan perintah siap-salin ada di `SETUP-GUIDE.md` pada repo.

#	FASE	INTI PEKERJAAN
1	Push kode ke GitHub	<code>git init</code> && <code>git push</code> — repo ini belum jadi git repo
2	Siapkan 2 VPS	Staging & production terpisah; <code>scripts/provision-vps.sh</code> instal Docker + buat user <code>deploy</code>
3	Isi <code>.env</code> di VPS	<code>GHCR_REPO</code> , <code>IMAGE_TAG=latest</code> , <code>JWT_SECRET</code> unik per environment
4	Token GHCR	Buat PAT <code>read:packages</code> , uji <code>docker login</code> manual sekali
5	GitHub Secrets & Environments	Isi tabel Bagian 6 di atas, aktifkan required reviewer production
6	DNS Cloudflare	A record <code>jeonme.com</code> , <code>www</code> , <code>staging.jeonme.com</code>
7	Sertifikat TLS production	<code>scripts/issue-certbot-cert.sh</code> — sekali saja, lalu jadwalkan renewal via cron VPS
8	Deploy pertama ke staging	Push ke <code>main</code> → otomatis → verifikasi <code>/api/health</code>
9	Deploy pertama ke production	<code>git tag v0.1.0</code> && <code>git push origin v0.1.0</code> → approve → verifikasi
10	Alur harian & rollback	Lihat Bagian 8 & 9 di bawah

8. Checklist Sebelum Rilis Production Pertama Kali

- ☐ `.env` production pakai kredensial live PSP (bukan sandbox), tidak pernah masuk repo
- ☐ `GHCR_REPO` di `.env` VPS staging & production sudah diisi `owner/repo` sungguhan
- ☐ Secret `GHCR_USERNAME` + `GHCR_PAT` dibuat, `docker login` dari VPS sudah diuji manual

- ☐ Endpoint webhook PSP diarahkan ke domain production, signature verification aktif
- ☐ Perintah migrasi (`./api migrate up`) sudah teruji idempoten di staging, dijalankan otomatis oleh pipeline
- ☐ Backup otomatis database (`pg_dump` terjadwal cron + upload object storage) sudah berjalan
- ☐ Sertifikat Certbot terbit, server block 443 aktif, cron renewal terjadwal
- ☐ Health check endpoint dites gagal→sukses
- ☐ GitHub Environment "production" punya required reviewers
- ☐ Smoke test transaksi dengan uang sungguhan (nominal kecil) sebelum go-live publik

9. Alur Harian & Rollback Darurat

Alur harian: branch fitur dari `main` → PR + review → merge (squash disarankan) → auto-deploy staging → QA → tag rilis → approve → production. Pantau health check + log setelah tiap deploy.

Rollback darurat (belum otomatis — item L2 di atas) via `scripts/rollback.sh` di VPS terkait:

```
cd /opt/jeonme
./scripts/rollback.sh docker-compose.prod.yml <sha-commit-rilis-sebelumnya>
```

Skrip ini hanya mengganti image, **tidak** menjalankan migrasi turun. Kalau rilis bermasalah sudah menjalankan migrasi skema yang tidak backward-compatible, rollback image saja tidak cukup — perlu keputusan manual.